BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Practical No: 1

Division: C Batch: 2

Roll no: 3090358 PRN No: 12410037

Name of Student: Radhyeshyam Rameshwar Lahoti

Subject Name : Deep Learning

Problem Statement:

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalisation, train-test splitting, and data visualisation on a sample dataset.

Algorithm :

Libraries Used:

1. TensorFlow (tensorflow)
   - Used for building, training, and evaluating the Artificial Neural Network (ANN).
   - Provides functions for loading the dataset, creating layers, compiling, training, and testing the model.

2. Matplotlib (matplotlib.pyplot)
   - Used for data visualisation.
   - Displays sample Fashion MNIST images using imshow().

Dataset Description: Fashion MNIST

Fashion MNIST is an image classification dataset containing grayscale images of clothing items.

- It contains 70,000 images:
  - 60,000 training images
  - 10,000 testing images
- Each image has a size of 28 × 28 pixels.

- Pixel values range from 0–255 and are normalised to 0–1 before training.
- The dataset has 10 classes:
  T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle Boot.

Input: 28×28 grayscale image
Output: One of 10 clothing categories.

Steps:

Import Required Libraries
- Import TensorFlow for building the neural network.
- Import Matplotlib for visualization.

Load the Dataset
- Load the Fashion MNIST dataset using TensorFlow.
- Split the data into training and testing sets.

Preprocess the Data
- Normalize image pixel values from **0–255 to 0–1**.
- Prepare the images for model training.

Visualize the Data
- Display sample images from the dataset using Matplotlib.

Build the Neural Network Model
- Create a Sequential model.
- Add a Flatten layer to convert images into a 1D array.
- Add a Dense hidden layer with ReLU activation.
- Add an output layer with Softmax activation for classification.

Compile the Model
- Select Adam optimizer.
- Use Sparse Categorical Crossentropy as the loss function.
- Set accuracy as the evaluation metric.

Train the Model
- Train the neural network using training images and labels.
- Run training for 10 epochs.

Evaluate the Model
- Test the model using test images.
- Calculate and display the accuracy.

Display Results
- Print the final test accuracy of the trained model.

Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt

# Get data and split into train and test datasets
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()

#Normalisation of data
train_images=train_images/255.0
test_images=test_images/255.0

# Visualizing data
print(train_images.shape)
print(test_images.shape)
print(train_labels)

# Display first image
plt.imshow(train_images[8], cmap='gray')
plt.show()

# Defining neural network model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# Training the model
my_model.fit(train_images, train_labels, epochs=10)

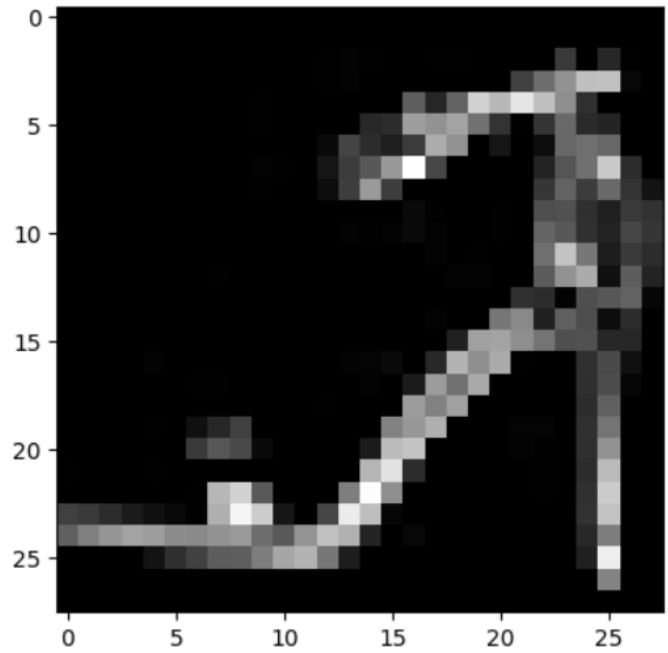
# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

# Saving the model
my_model.save('my_fashion_mnist_model.keras')
```

Output :

Visualizing data:

```
(60000, 28, 28)
(10000, 28, 28)
[9 0 0 ... 3 0 5]
```



Training of the dataset with 10 epochs:

```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.8249 - loss: 0.4998
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8637 - loss: 0.3761
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8774 - loss: 0.3375
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8840 - loss: 0.3144
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8912 - loss: 0.2954
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8957 - loss: 0.2814
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9003 - loss: 0.2689
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9036 - loss: 0.2581
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9075 - loss: 0.2478
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9111 - loss: 0.2400
<keras.src.callbacks.history.History at 0x7b384df65bb0>
```

Accuracy of testing data:

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8864 - loss: 0.3273
Test accuracy:  0.8863999843597412
```